TALLINN UNIVERSITY OF TECHNOLOGY
School of Information Technologies

Kristofer Petermann IADB232897

"Community-based restaurant ordering platform"

Web Applications with C# project

Supervisor: Andres Käver

Tallinn 2026

Author’s declaration of originality

I hereby certify that I am the sole author of this project. All the used materials, references to the literature and the work of others have been referred to. This project has not been presented for examination anywhere else.

Author: Kristofer Petermann

26.02.2026

Table of contents

1 Introduction 4

2 Analysis 5

2.1 Problem statement 5

2.2 Proposed Solution 5

2.3 System Objectives 6

2.4 Motivation 6

3 System Overview 7

3.1 Customers 7

3.2 Restaurant Administrators 7

3.3 System Administrator 7

4 Entity-Relationship Diagram 8

4.1 Identity Layer 8

4.2 Domain Layer 8

5 User Flow 10

5.1 Customer Positive Flow 10

5.2 Restaurant Administrator Flow 10

1 Introduction

Digital food ordering platforms have become an essential part of modern urban infrastructure. Customers increasingly expect convenient access to restaurant services through online systems, while restaurants rely on digital tools to remain competitive. However, many existing commercial food delivery platforms operate on a commission-based model, where restaurants pay a significant percentage of each transaction.

In this project I will start the development of a community-based restaurant ordering platform focused on pickup orders. This system enables restaurants to manage their menus and receive customer orders directly through a centralized web application. Unlike large commercial delivery platforms, the proposed system emphasizes transparency, reduced operational costs, and simplified logistics by excluding courier-based delivery functionality.

The primary objective of this project is to design and implement a relational database-driven ASP.NET MVC web application that demonstrates structured data modeling, transactional integrity, and role-based access control within a realistic business domain.

2 Analysis

2.1 Problem statement

Small and independent restaurants often face significant financial pressure when using large third-party ordering platforms. Commission fees typically range between 20% and 35% per order, reducing already narrow profit margins. In addition to financial constraints, restaurants frequently lose control over customer data and brand identity when operating solely through external platforms.

Restaurants that avoid third-party systems often rely on manual order intake methods, such as phone calls. In my opinion this approach introduces multiple inefficiencies and risks:

- Human error in recording orders
- Miscommunication between staff and customers
- Lack of structured order history
- Absence of integrated data analytics
- Difficulty in tracking operational performance

Manual systems also limit scalability and make it difficult to maintain consistent service quality during peak hours.

2.2 Proposed Solution

The solution is a web-based restaurant ordering platform that supports direct customer-to-restaurant interaction. The system allows restaurants to register, manage menus, define opening hours, and process customer pickup orders within a structured digital environment.

Customers can browse restaurants, view categorized menus, place pickup orders, and leave reviews. Orders are stored in a normalized relational database structure that ensures

data integrity and historical consistency, including preservation of item prices at the time of purchase.

The system intentionally excludes courier logistics to maintain a focused scope while still demonstrating complex relational modeling and transactional processing.

2.3 System Objectives

The main objectives of the system are:

- To provide a structured and normalized relational database model
- To demonstrate many-to-many relationship modeling through contextual restaurant administration
- To implement transactional order processing with order and order-line separation
- To support role-based authorization using ASP.NET Core Identity
- To preserve historical pricing data within order rows
- To design a scalable architecture suitable for extension

2.4 Motivation

Digital solutions have become an essential part of everyday services, including the restaurant industry. While large platforms dominate the market, smaller restaurants often struggle to maintain independence and financial sustainability when relying on external ordering systems.

The motivation for this project is to explore how a simplified and transparent online ordering platform could support small restaurants in managing their operations more independently. By focusing on pickup orders and avoiding delivery logistics, the system remains practical and realistic while still addressing real operational needs.

This project also provides an opportunity to design and implement a structured web application that reflects real business workflows. The restaurant ordering domain was selected because it represents a familiar and relevant context in which data management, order processing, and user interaction naturally come together.

3 System Overview

The system consists of three primary user groups:

3.1 Customers

Customers can:

- Register and authenticate using ASP.NET Core Identity
- Browse restaurants
- View categorized menus
- Place pickup orders
- Review restaurants

3.2 Restaurant Administrators

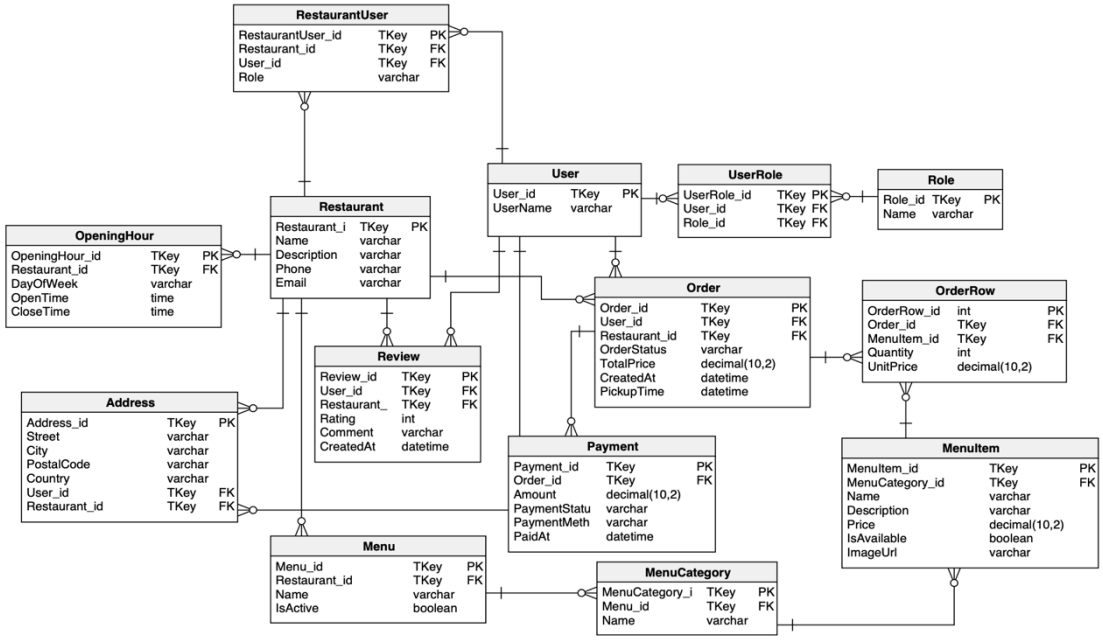
Restaurant administrators are associated with specific restaurants through a contextual many-to-many relationship. They can:

- Manage restaurant information
- Define opening hours
- Create and modify menus and categories
- Add, update, or deactivate menu items
- View and process incoming orders

3.3 System Administrator

The system administrator manages platform-level configurations and user oversight.

4 Entity-Relationship Diagram



The system architecture is divided into two layers:

4.1 Identity Layer

Authentication and global authorization are handled using ASP.NET Core Identity.

This includes the following framework-provided entities:

- User
- Role
- UserRole

4.2 Domain Layer

The custom domain model consists of the following entities:

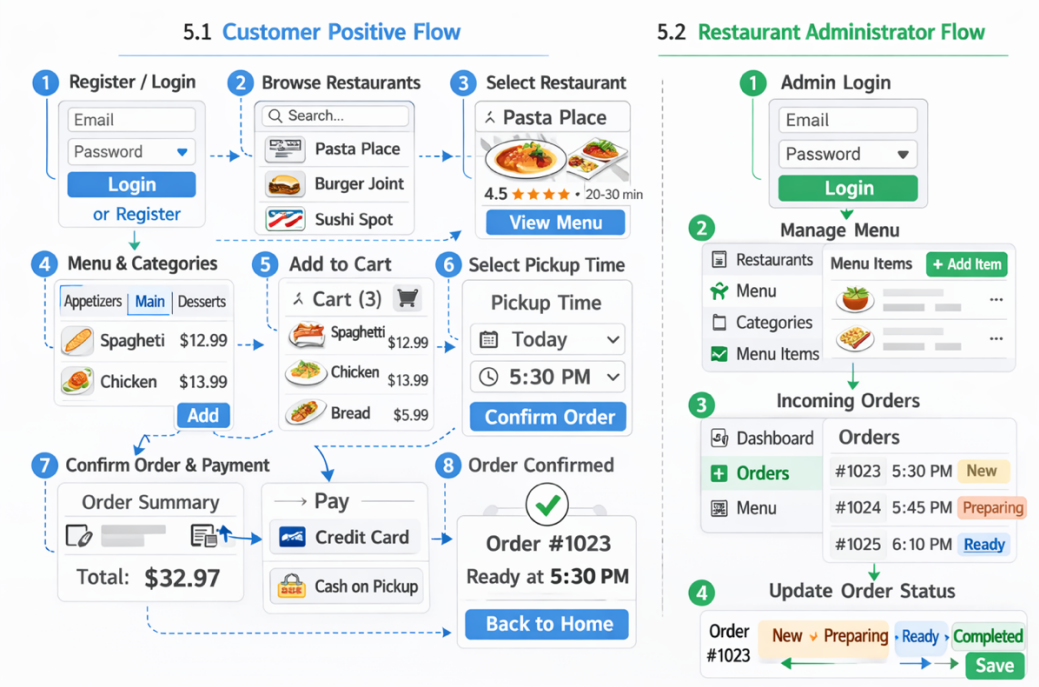
- Restaurant
- OpeningHour
- Address
- Menu

- MenuCategory
- MenuItem
- Order
- OrderRow
- Payment
- Review

Additionally, RestaurantUser is implemented connecting users and restaurants.

The Order and OrderRow separation ensure proper transactional modeling, allowing a single order to contain multiple menu items. The UnitPrice field in OrderRow preserves historical pricing information. Referential integrity is enforced through foreign key constraints, and a composite uniqueness constraint prevents duplicate menu items within a single order.

5 User Flow



5.1 Customer Positive Flow

1. User registers or logs in.
2. User browses available restaurants.
3. User selects a restaurant.
4. User views menu categories and menu items.
5. User adds items to the order.
6. User selects a pickup time.
7. User confirms the order.
8. Payment is registered.
9. Order status updates accordingly.

5.2 Restaurant Administrator Flow

1. Administrator logs in.
2. Administrator manages menu data.
3. Administrator receives incoming orders.
4. Administrator updates order status.